NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University

LAB REPORT FOR COMPILER DESIGN

notes.arpanadhikari7.com.np

LAB REPORT 1,2,3

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BECE (6th SEM)

Submitted to:

Subash Manandhar

Available on notes.arpanadhikari7.com.np

QSN 1: PROGRAM TO CHECK WHETHER INPUT IS AN IDENTIFIER

THEORY

In programming languages, an **identifier** is the name given to variables, functions, arrays, or other user-defined items. Identifiers must follow specific lexical rules defined by the compiler.

Rules for Identifiers

- Must begin with a **letter (a–z, A–Z)** or an **underscore (_)**
- Cannot start with a digit
- Can contain **letters, digits, and underscores**
- No special characters like @, #, $, %, etc. are allowed
- Should not be a reserved keyword

Concept Used

This program simulates a **lexical analyzer phase** of a compiler, where tokens are validated based on syntax rules. Functions like isalpha() and isalnum() help classify characters.

//Code

```
#include <iostream>
#include <cctype>
#include <string>
using namespace std;

bool isIdentifier(string str) {
    if (str.empty() || (!isalpha(str[0]) && str[0] != '_')) return false;
    for (size_t i = 1; i < str.length(); i++) {
        if (!isalnum(str[i]) && str[i] != '_') return false;
    }
    return true;
}

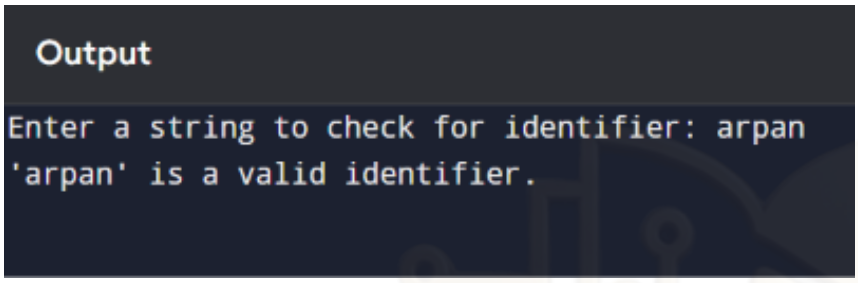
int main() {
    string input;
    cout << "Enter a string to check for identifier: ";
    cin >> input;
```


Available on notes.arpanadhikari7.com.np

```
    if (isIdentifier(input))
        cout << "'" << input << "' is a valid identifier." << endl;
    else
        cout << "'" << input << "' is NOT a valid identifier." << endl;

    return 0;
}
```

//Output



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 2: PROGRAM TO CHECK WHETHER INPUT IS A KEYWORD

Theory

A **keyword** is a reserved word in a programming language that has a predefined meaning and cannot be used as an identifier.

Examples of Keywords

int, float, if, else, return, while, for, class, struct

Concept Used

The program uses a **set data structure** to store predefined keywords. It then checks whether the given input exists in that set. This simulates how a compiler distinguishes keywords during lexical analysis.

//Code

```
#include <iostream>
#include <string>
#include <set>
using namespace std;

bool isKeyword(string str) {
    set<string> keywords = {
        "int", "float", "char", "double", "if", "else", "while", "do", "for",
        "return", "switch", "case", "break", "continue", "void", "class", "struct"
    };
    return keywords.find(str) != keywords.end();
}

int main() {
    string input;
    cout << "Enter a string to check for keyword: ";
    cin >> input;



    if (isKeyword(input))
```

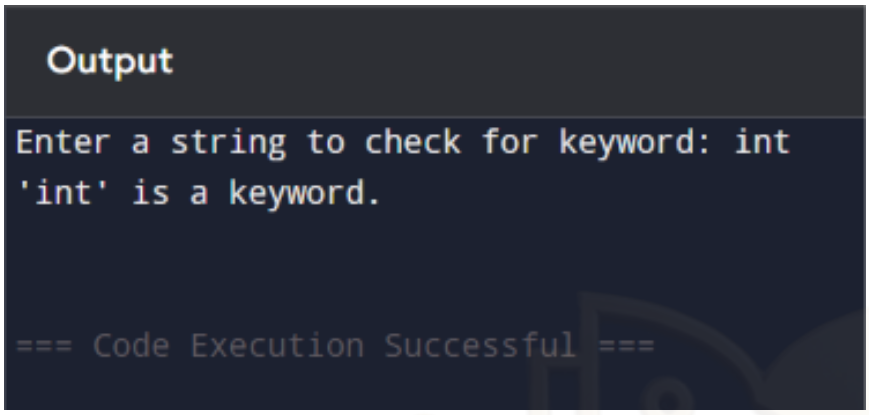
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
        cout << "'" << input << "' is a keyword." << endl;
    else
        cout << "'" << input << "' is NOT a keyword." << endl;

    return 0;
}
```

//Output



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 3: PROGRAM TO CHECK WHETHER INPUT IS AN OPERATOR

Theory

An **operator** is a symbol used to perform operations on variables and values.

Types of Operators

- Arithmetic: + - * / %
- Relational: < > <= >= == !=
- Logical: && ||
- Assignment: =

Concept Used

The program stores all valid operators in a **set** and checks if the given input matches any of them. This is part of token classification in the lexical analysis phase.

//Code

```
#include <iostream>
#include <string>
#include <set>
using namespace std;

bool isOperator(string str) {
    set<string> operators = {
        "+", "-", "*", "/", "%", "=", "==", "!=", "<", ">", "<=",
        ">=", "&&", "||"
    };
    return operators.find(str) != operators.end();
}

int main() {
    string input;
    cout << "Enter a string to check for operator: ";
    cin >> input;
```

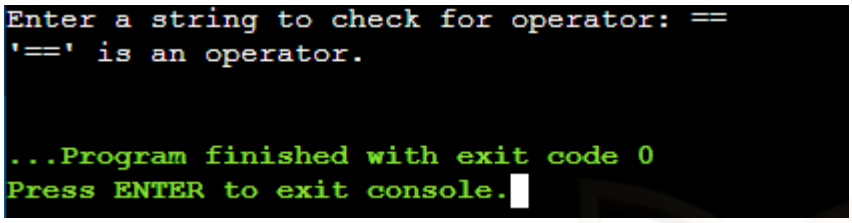
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    if (isOperator(input))
        cout << "'" << input << "' is an operator." << endl;
    else
        cout << "'" << input << "' is NOT an operator." << endl;

    return 0;
}
```

//Output



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 4: CONSTANT CHECK

Theory

A **constant** is a fixed value that does not change during program execution. Constants can be classified into:

- **Integer constants** → 10, -25
- **Floating-point constants** → 3.14, -0.5

Rules

- May start with + or -
- Can contain digits and at most **one decimal point**
- Cannot contain alphabets or special symbols

Concept

The program checks:

- Whether all characters are digits
- Whether the decimal point appears only once

This simulates how compilers validate numeric tokens.

//Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char str[100];
    int i, len;
    int isInteger = 1, isFloat = 1;

    printf("Enter input: ");
    scanf("%s", str);

    len = strlen(str);
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
if (str[0] == "" && str[len - 1] == "") {
    printf("%s is a String Constant.\n", str);
}


else if (str[0] == '\'' && str[len - 1] == '\'' && len == 3) {
    printf("%s is a Character Constant.\n", str);
}

else {
    for (i = 0; str[i] != '\0'; i++) {
        if (!isdigit(str[i])) {
            isInteger = 0;
            break;
        }
    }

    if (isInteger) {
        printf("%s is an Integer Constant.\n", str);
    }
    else {
        int dotCount = 0;

        for (i = 0; str[i] != '\0'; i++) {
            if (str[i] == '.') {
                dotCount++;
            }
            else if (!isdigit(str[i])) {
                isFloat = 0;
                break;
            }
        }

        if (isFloat && dotCount == 1) {
```

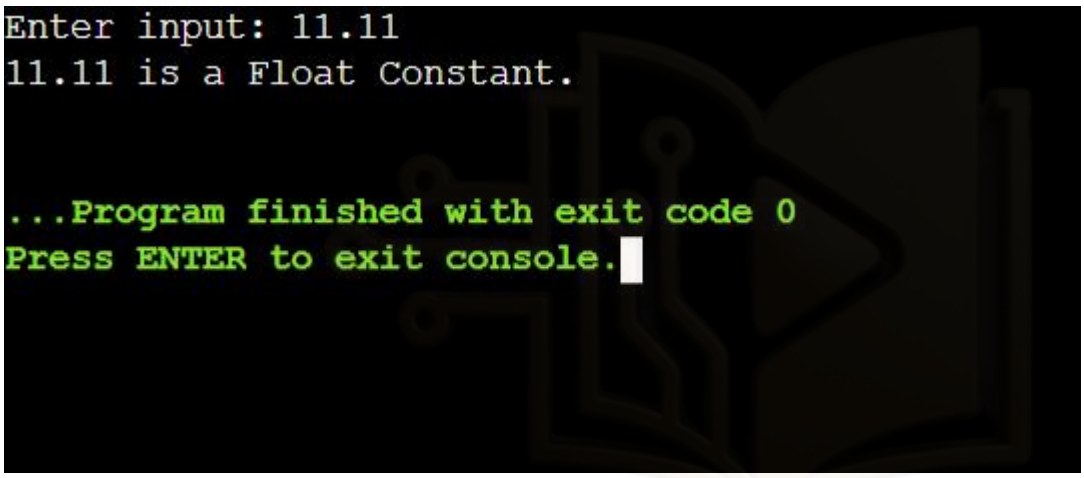
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
            printf("%s is a Float Constant.\n", str);
        }
        else {
            printf("%s is NOT a Constant.\n", str);
        }
    }
  }

  return 0;
}
```

//Output



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 5: COMMENT DETECTION

Theory

Comments are **non-executable statements** used to improve code readability and documentation.

Types of Comments

1. **Single-line comment** → starts with //
2. **Multi-line comment** → enclosed within /* ... */

Concept

The program identifies comments by checking:

- Starting characters (// or /*)
- Ending characters (*/ for multi-line)

This is part of lexical analysis where comments are ignored during token generation.

//Code

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[200];

    printf("Enter input: ");
    fgets(str, sizeof(str), stdin);

    int len = strlen(str);

    if (str[len - 1] == '\n') {
        str[len - 1] = '\0';
        len--;
    }

    if (str[0] == '/' && str[1] == '/') {
        printf("Single Line Comment\n");
```

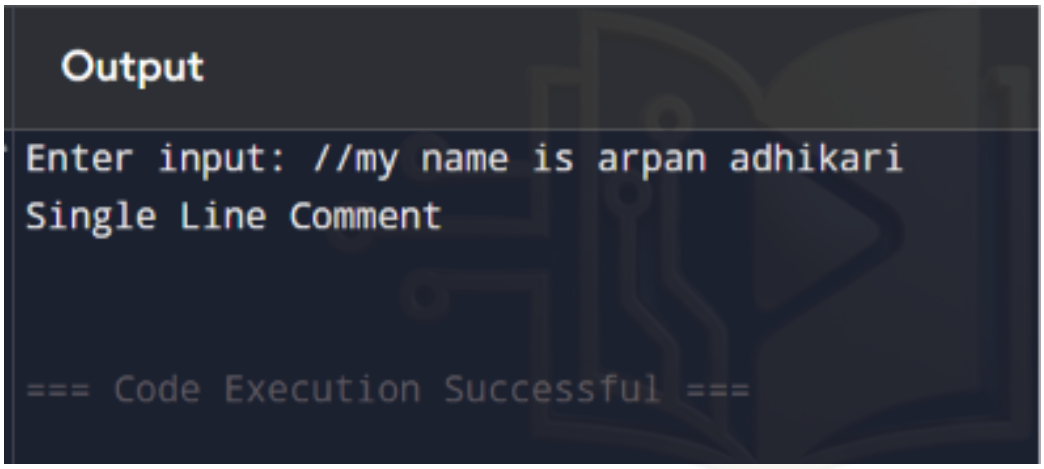
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    }
    else if (str[0] == '/' && str[1] == '*' &&
             str[len - 2] == '*' && str[len - 1] == '/') {
        printf("Multi Line Comment\n");
    }
    else {
        printf("Not a Comment\n");
    }

    return 0;
}
```

//Output



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 6: TOKENIZATION

Theory

Tokenization is the process of breaking a program into smaller units called **tokens**.

Types of Tokens

- Keywords
- Identifiers
- Operators
- Constants
- Punctuation symbols

Concept

The program uses string processing (or regex) to extract tokens from a source string. This step is the **first stage of compilation**.

Example:

int a = 10;

Tokens → int, a, =, 10, ;

//Code

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int isKeyword(char str[]) {
    char *keywords[] = {
        "void", "main", "int", "float", "char",
        "if", "else", "while", "for", "return", "printf"
    };

    int n = sizeof(keywords) / sizeof(keywords[0]);

    for (int i = 0; i < n; i++) {
        if (strcmp(str, keywords[i]) == 0)
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
            return 1;
        }
        return 0;
}

int main() {
    char str[] =
        "void main(){int a,b; a=a+b; printf(\"Hello!\"); "
        "a=5; b=10; a=a+b; return 0;}";

    char token[100];
    int i = 0, j;

    while (str[i] != '\0') {


        if (isspace(str[i])) {
            i++;
            continue;
        }

        if (str[i] == '"') {
            j = 0;
            token[j++] = str[i++];

            while (str[i] != '"' && str[i] != '\0')
                token[j++] = str[i++];

            if (str[i] == '"')
                token[j++] = str[i++];

            token[j] = '\0';

            printf("%s - Constant\n", token);
        }
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    else if (isalpha(str[i]) || str[i] == '_') {
        j = 0;

        while (isalnum(str[i]) || str[i] == '_')
            token[j++] = str[i++];

        token[j] = '\0';

        if (isKeyword(token))
            printf("%s - Keyword\n", token);
        else
            printf("%s - Identifier\n", token);
    }

    else if (isdigit(str[i])) {
        j = 0;

        while (isdigit(str[i]))
            token[j++] = str[i++];

        token[j] = '\0';

        printf("%s - Constant\n", token);
    }

    else if (strchr("=+-*/", str[i])) {
        printf("%c - Operator\n", str[i]);
        i++;
    }

    else {
        i++;
    }
}
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    return 0;
}
```

//Output

```
void - Keyword
main - Keyword
int - Keyword
a - Identifier
b - Identifier
a - Identifier
= - Operator
a - Identifier
+ - Operator
b - Identifier
printf - Keyword
"Hello!" - Constant
a - Identifier
= - Operator
5 - Constant
b - Identifier
= - Operator
10 - Constant
a - Identifier
= - Operator
a - Identifier
+ - Operator
b - Identifier
return - Keyword
0 - Constant


...Program finished with exit code 0
Press ENTER to exit console.
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 7: COUNT LINES AND COMMENTS

Theory

Analyzing source code often requires counting:

- Total number of lines
- Number of comment lines

Concept

The program:

1. Reads code line-by-line using getline()
2. Increments line counter
3. Checks for comment patterns (// or /*)

This is useful in:

- Code analysis tools
- Metrics calculation
- Preprocessing stage

//Code

```
#include <stdio.h>

int main() {
    char code[1000];
    int i = 0;

    int lines = 1;
    int single_comments = 0;
    int multi_comments = 0;

    int in_single = 0;
    int in_multi = 0;

    printf("Enter the program (end with ~):\n");

    while ((code[i] = getchar()) != '~') {
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    i++;
  }
  code[i] = '\0';

  for (int j = 0; code[j] != '\0'; j++) {

    if (code[j] == '\n')
      lines++;

    if (code[j] == '/' && code[j+1] == '/' && !in_multi) {
      single_comments++;
      in_single = 1;
    }

    if (code[j] == '/' && code[j+1] == '*' && !in_single) {
      multi_comments++;
      in_multi = 1;
    }

    if (code[j] == '\n' && in_single) {
      in_single = 0;
    }

    if (code[j] == '*' && code[j+1] == '/' && in_multi) {
      in_multi = 0;
    }
  }

  printf("\nTotal Lines = %d\n", lines);
  printf("Single-line Comments = %d\n", single_comments);
  printf("Multi-line Comments = %d\n", multi_comments);

  return 0;
}
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

//Output

Output

```
Enter the program (end with ~):
int main(){
print('My name is Arpan');
//comment
}~

Total Lines = 4
Single-line Comments = 1
Multi-line Comments = 0
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 8: PATTERN RECOGNITION (REGULAR EXPRESSIONS)

Theory

Regular expressions (Regex) are used to describe patterns in strings. In compiler design, regex is used to define tokens.

Given Patterns

- a*b → zero or more a followed by b
- a+ba → one or more a followed by ba
- (a+b)* → any combination of a and b

Concept

The program uses regex_match() to compare input strings with defined patterns. This is directly related to **finite automata theory**.

//Code

```
#include <stdio.h>
int main() {
   char str[100];
   int i, valid = 1;

   printf("---------------------------------\n");
   printf("Name: Sanjog Sharma -231336\n");
   printf("---------------------------------\n");

   printf("Enter string: ");
   scanf("%s", str);

   for (i = 0; str[i] != '\0'; i++) {
      if (str[i] != 'a' && str[i] != 'b') {
         valid = 0;
         break;
      }
   }
```

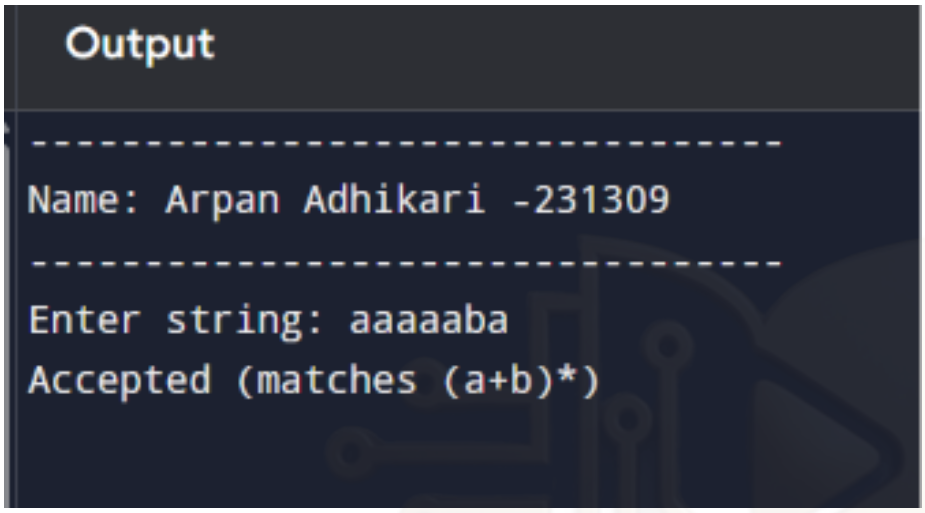
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    if (valid)
        printf("Accepted (matches (a+b)*)\n");
    else
        printf("Rejected\n");

    return 0;
}
```

//Output



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 9: DFA SIMULATION

Theory

A **Deterministic Finite Automaton (DFA)** is a mathematical model used to recognize patterns in strings.

Components of DFA

- States
- Input alphabet
- Transition function
- Start state
- Accept state

Concept

The program simulates DFA behavior by:

- Checking if input matches identifier rules
- Checking if input is numeric (constant)
- Checking operator patterns

This demonstrates how compilers use automata for token recognition.

//Code

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int isIdentifier(char str[]) {
    if (!(isalpha(str[0]) || str[0] == '_'))
        return 0;

    for (int i = 1; str[i] != '\0'; i++) {
        if (!(isalnum(str[i]) || str[i] == '_'))
            return 0;
    }
    return 1;
}
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
int isConstant(char str[]) {
    for (int i = 0; str[i] != '\0'; i++) {
        if (!isdigit(str[i]))
            return 0;
    }
    return 1;
}

int isOperator(char str[]) {
    char *ops[] = {"+","-","*","/","%","=","==","!=","<",">","<=",">="};
    int n = 12;

    for (int i = 0; i < n; i++) {
        if (strcmp(str, ops[i]) == 0)
            return 1;
    }
    return 0;
}

int main() {
    char str[50];

    printf("Enter input: ");
    scanf("%s", str);

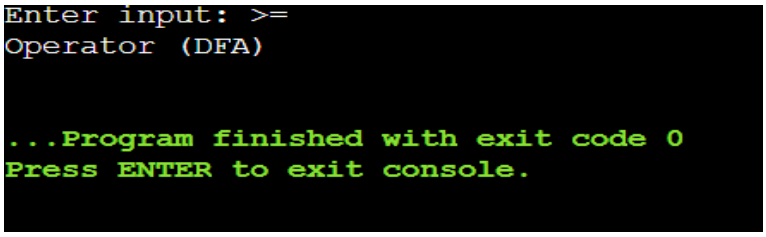
    // DFA: sequential checking (one path)
    if (isIdentifier(str))
        printf("Identifier (DFA)\n");
    else if (isConstant(str))
        printf("Constant (DFA)\n");
    else if (isOperator(str))
        printf("Operator (DFA)\n");
    else
        printf("Invalid String\n");
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    return 0;
}
```

//Output



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

QSN 10: LEXICAL ANALYZER

Theory

A **Lexical Analyzer** (Scanner) is the first phase of a compiler. It converts source code into tokens.

Functions of Lexical Analyzer

1. Reads input program
2. Removes whitespace and comments
3. Identifies tokens
4. Sends tokens to syntax analyzer

Token Classification

- Keyword
- Identifier
- Operator
- Constant

Concept

The program splits a string into tokens and classifies each token based on predefined rules.

Example:

int x = 10

→ int (Keyword)

→ x (Identifier)

→ = (Operator)

→ 10 (Constant)

notes.arpanadhikari7.com.np

//Code

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

int main() {
```


Available on notes.arpanadhikari7.com.np

```
char str[50];
int i;

printf("Enter input: ");
scanf("%s", str);

// Parallel states (NFA)
int isId = 1;
int isNum = 1;
int isOp = 0;

// Operator check
char *ops[] = {"+","-","*","/","%","=","==","!=","<",">","<=",">="};
int n = 12;

for (i = 0; i < n; i++) {
    if (strcmp(str, ops[i]) == 0) {
        isOp = 1;
        break;
    }
}

// Simultaneous checking
for (i = 0; str[i] != '\0'; i++) {

    if (!(isalnum(str[i]) || str[i] == '_'))
        isId = 0;

    if (!isdigit(str[i]))
        isNum = 0;
}

if (!(isalpha(str[0]) || str[0] == '_'))
    isId = 0;
```

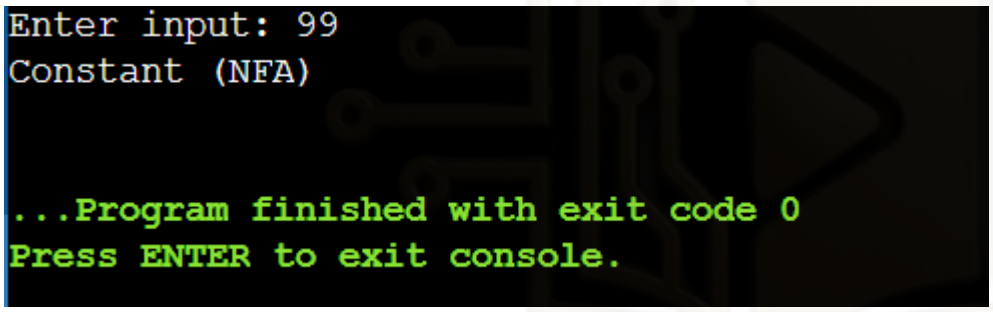
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

```
    // Multiple accept states
    if (isId)
        printf("Identifier (NFA)\n");
    else if (isNum)
        printf("Constant (NFA)\n");
    else if (isOp)
        printf("Operator (NFA)\n");
    else
        printf("Invalid String\n");

    return 0;
}
```

//Output



notes.arpanadhikari7.com.np

CONCLUSION

These experiments demonstrate the fundamental concepts of lexical analysis, including token identification, pattern recognition, and DFA simulation, which are essential phases in compiler design.

Available on notes.arpanadhikari7.com.np